2025 32nd Asia-Pacific Software Engineering Conference (APSEC) | 979-8-3315-6653-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/APSEC66846.2025.00017

TRACE: Test Repair via Agent-based Context Extraction with LLMs

1st Jingxiang Tu
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
tujingxiang23@nudt.edu.cn

2nd Bo Lin*
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
linbo19@nudt.edu.cn

3rd Yihao Qin
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
yihaoqin@nudt.edu.cn

4th Shangwen Wang
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
wangshangwen13@nudt.edu.cn

5th Liqian Chen
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
lqchen@nudt.edu.cn

6th Xiaoguang Mao*
College of Computer Science and Technology
National University of Defense Technology
Changsha, China
xgmao@nudt.edu.cn

***Abstract*—As software evolves, test code must be co-maintained to ensure quality, but it often becomes obsolete, leading to failures that mislead developers and increase maintenance overhead. While recent Large Language Model (LLM)-based approaches show promise for repairing obsolete tests, their effectiveness is constrained by a critical challenge: providing comprehensive, repository-level context without overwhelming the models' input limits. Fixed retrieval strategies often fail to capture the diverse dependencies required for complex repairs. In this paper, we present TRACE, a retrieve-agent-based repository-level test repair method. The TRACE selectively retrieves crucial context by analyzing (1) class-level structures to understand internal changes, (2) caller methods to capture real-world usage patterns, and (3) related files along the call graph to trace transitive dependencies. This multi-faceted context provides the LLM with a precise and concise understanding of the necessary code environment. We evaluated TRACE on a dataset of real-world Java test updates, where it demonstrated superior performance compared to existing state-of-the-art baselines. Our results confirm that a structured, adaptive retrieval process is key to unlocking the full potential of LLMs for automated test maintenance.**

***Index Terms*—Test Repair, Software Maintenance, LLM**

I. INTRODUCTION

Software testing is a critical phase of the software development lifecycle, ensuring the quality, reliability, and stability of software systems [1, 2, 3, 4, 5]. In the complex process of software development, test engineers design and execute numerous test cases to verify that the system meets expected specifications and to ensure that code modifications do not introduce new defects or break existing functionality. When production code passes all tests, it suggests that the changes align with the intended objectives; conversely, test failures may indicate defects that require developer attention. Pinto et al. [6] conducted an empirical study on test suite evolution by collecting multiple versions of code from various open-source projects. Their research indicated that test code changes account for 29.3% of test suite evolution in actual project development. This highlights the importance of updating and maintaining test code to ensure the effectiveness of the test suite. However, as production code evolves, test cases often become obsolete, leading to false positives and imposing significant burdens on development teams. For example, Daka et al. [7] surveyed 225 software developers across 29 countries, covering users of different programming languages. Their study revealed that 52.8% of test case execution failures were due to obsolete test cases that no longer met current requirements. These inaccuracies mislead developers, increase maintenance overhead, and delay project delivery. Therefore, effectively updating test code to mitigate the issues caused by obsolete test cases has become an urgent problem in the field of software testing.

To mitigate the consequences of obsolete test cases, existing studies have explored various approaches to automatic test code evolution. For instance, Marsavina et al. [8] identified common co-evolution patterns between production and test code. However, the co-evolution of production and test code exhibits significant diversity in real-world software projects, making it difficult to encapsulate this phenomenon with simplistic patterns. In recent years, the rapid advancement

*Corresponding authors

979-8-3315-6653-1/25/$31.00 ©2025 IEEE
DOI 10.1109/APSEC66846.2025.00017

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

of machine learning and deep learning has led to efforts focusing on using these technologies to address the challenges of test code evolution. Notable approaches include machine learning-based identification of obsolete test cases [9] and fine-tuned CodeT5 [10] models for detecting and updating obsolete test code [11]. These approaches leverage the powerful capabilities of machine learning and deep learning to identify and repair various test code issues. Although these methods have shown potential, their effectiveness remains limited due to the complexity and dynamic nature of test code evolution. The advent of Large Language Models (LLMs), with their sophisticated code comprehension and generation capabilities, has opened a new frontier for this problem. A representative work, SYNTER [12], employs an LLM-based framework that uses focal method changes and obsolete test code, augmented with contextual information extracted via predefined heuristics. However, these fixed retrieval strategies are often insufficient. The context required to repair a test can be subtle and non-local, involving inherited behaviors, utility functions, or global configurations spread across multiple files-dependencies that are difficult to anticipate with static rules. Compounding this challenge is a fundamental limitation of LLMs: their reasoning ability degrades with longer input contexts [13, 14]. Simply providing a large volume of code is therefore counterproductive, as it can overwhelm the model and obscure critical information. This reveals a central tension for LLM-based test repair: while LLMs possess powerful reasoning abilities, their effectiveness is contingent on receiving precise, relevant, yet concise contextual input. The key challenge, therefore, is to develop retrieval mechanisms that can adaptively and efficiently identify this essential context from a large repository.

To address this critical challenge of context retrieval for test repair, we propose TRACE, a repository-level approach that leverages LLMs to automatically generate updated test cases. Unlike existing techniques that rely on local context or fixed heuristics, TRACE systematically gathers relevant information from the entire project repository. It overcomes the limitations of prior work by introducing a dynamic, multi-stage retrieval agent that selectively gathers the most salient context. Specifically, the agent retrieves: (1) class-level context to capture internal structural changes and inheritance hierarchies, (2) caller methods to understand real-world usage patterns of the modified code, and (3) transitively related files identified through the project's call graph to trace non-local dependencies. By strategically synthesizing these complementary sources of information, TRACE provides the LLM with a concise yet comprehensive understanding of the required changes, enabling it to generate test code that is not only syntactically correct but also semantically consistent with the evolved codebase.

To summarize, our work makes the following contributions:

- We conduct an empirical study to investigate the essential contextual information required for updating obsolete tests. Based on our findings, we design a repository-level context retrieval agent to effectively and efficiently extract this critical information.
- We propose TRACE, an automated approach for retrieving relevant contextual information from the project repository to facilitate test code updates. TRACE enables comprehensive modernization of obsolete test cases by dynamically analyzing repository-wide dependencies and semantic relationships.
- The experimental results demonstrate that TRACE effectively corrects obsolete test cases and exhibits more effectively performance compared to existing methods. We also open source the artifacts of this study at https://zenodo.org/records/15729689 to facilitate the future studies.

II. RELATED WORK

A. Large Language Models

LLMs have made significant progress in the fields of natural language processing and code related tasks [15, 16, 17, 18, 19, 20, 21]. LLMs such as GPT-4 [22], DeepSeek-Coder [23], which are trained on vast amounts of data, have demonstrated remarkable capabilities in language understanding and generation. These models have not only excelled in traditional NLP tasks such as text generation and translation but have also been widely applied in software engineering domains, including code generation and vulnerability detection [24, 25].

B. Co-evolution of Production and Test Code

The co-evolution of production and test code is crucial for maintaining software quality throughout the software development lifecycle [26, 27, 28]. As production code evolves, corresponding test code must be updated to ensure that existing functionalities remain intact and new features are adequately tested. However, this synchronization is often challenging and time-consuming [29, 30, 31, 32].

Marsavina et al. [8] conducted an empirical study analyzing fine-grained co-evolution patterns between production and test code across five open-source systems. They identified six co-evolution patterns, such as the creation of test classes when new production classes are added and the deletion of test classes when production classes are removed. Their findings highlighted that well-tested projects are more likely to follow positive co-evolution patterns, emphasizing the importance of synchronized development. Building upon this, Wang et al. [9] performed a large-scale empirical study on 975 open-source Java projects to understand co-evolution practices. They observed that while co-evolution is common, many production code changes do not result in corresponding test code updates. To address this, they proposed SITAR, a machine-learning-based approach that predicts whether test code should be updated when production code changes. However, SITAR's reliance on naming conventions and manually summarized features limited its effectiveness. To overcome SITAR's limitations, Hu et al. [11] introduced CEPROT, a novel approach that identifies and automatically updates obsolete test cases based on production code changes. CEPROT leverages CodeT5 [10] model to learn from code changes and generate updated test cases. Their experiments demonstrated that CEPROT effectively identifies and updates test cases with high precision and

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

recall, significantly improving test coverage. Despite these advancements, existing methods often lack the capability to handle repository-level contextual information, which is crucial for accurately updating test code in large-scale projects. For instance, Liu et al. [12] proposed SYNTER, an approach that enhances LLM with static analysis and neural reranking to repair obsolete test cases caused by syntactic breaking changes. SYNTER constructs Test-Repair-Oriented Contexts by collecting relevant code information through static analysis and identifying the most pertinent contexts using neural rerankers. More recently, Chi et al. [33] introduced REACCEPT, a novel approach that leverages LLMs and dynamic validation to fully automate the co-evolution of production and test code. REACCEPT relies on experience-based prompt template generation, dynamic validation, and retrieval-augmented generation techniques to achieve automated co-evolution. Although both SYNTER and REACCEPT demonstrate improved performance, they still face challenges in capturing comprehensive repository-level context due to the input length limitations of LLMs.

While significant progress has been made in automating the co-evolution of production and test code, current approaches often fall short in effectively utilizing repository-level contextual information. Addressing this gap is essential for enhancing the accuracy and efficiency of test code updates in evolving software systems.

C. Test Generation Based on LLMs

In recent years, test generation leveraging LLMs has garnered significant attention [34, 35, 36, 37]. LLMs have demonstrated the capability to generate high-quality test cases from natural language descriptions, effectively covering complex code logic and boundary conditions [38, 39, 40]. For instance, TestBench [41] serves as a benchmark tool for evaluating the class-level test generation capabilities of LLMs, assessing the quality of generated test cases across various metrics, including syntactic correctness, compilation success, test correctness, code coverage, and defect detection rates. Despite these advancements, LLMs face challenges in generating test cases that are both readable and comprehensive in coverage. Issues such as lack of meaningful variable names and insufficient coverage of complex code branches persist. To address these shortcomings, researchers have proposed optimization methods that combine program analysis and search algorithms. For example, TELPA [42] enhances LLM-based test generation by extracting real usage scenarios and employing feedback mechanisms to iteratively refine test cases, thereby improving branch coverage.

While these methods focus on the initial generation of test cases, our work diverges by concentrating on the maintenance and repair of existing test code. Specifically, we address the challenge of updating obsolete or broken test cases to ensure their continued effectiveness in validating evolving production code. This approach is particularly pertinent in the context of software maintenance, where ensuring the usability and reliability of test suites over time is critical. Unlike test generation, which aims to create new tests, our focus is on preserving and enhancing the utility of existing tests amid ongoing repository changes.

III. BACKGROUND

In the context of software maintenance, test cases must evolve alongside production code to remain valid and effective. When the production code changes but the associated test code is not updated accordingly, the test case may become *obsolete*, leading to compilation errors, test failures, or semantic mismatches. We formally define the task of **test case update** as follows.

a) Obsolete Test Case.: Given a test case t and two consecutive versions of the production code P_{old} and P_{new}, we say that t is *obsolete* with respect to P_{new} if:

$$\text{behavior}(t, P_{new}) \neq \text{expected}(t)$$

where $\text{behavior}(t, P_{new})$ denotes the observed runtime behavior of executing t against P_{new}, and $\text{expected}(t)$ reflects the intended outcome as specified by the original test.

b) Problem Statement.: Given an obsolete test case t_{old}, updated production code P_{new}, and optional project repository $\mathcal{R}$ containing contextual information, the test case evolution problem requires generating t_{new} satisfying:

$$\begin{aligned} \text{compile}(t_{\text{new}}, P_{\text{new}}) &= \text{true} \\ \text{intent}(t_{\text{new}}) &\approx \text{intent}(t_{\text{old}}) \\ \text{behavior}(t_{\text{new}}, P_{\text{new}}) &= \text{expected}(t_{\text{new}}) \end{aligned}$$

where:

- $\text{compile}(t, P)$ ensures syntactic compatibility
- $\text{intent}(t)$ preserves original test semantics
- $\text{behavior}(t, P)$ validates runtime correctness

The updated test case should remain faithful to the original intent while being compatible with the updated production code.

IV. METHODOLOGY

A. Motivating Example

Fig. 1 presents a motivating example from the Apache/metron project that illustrates the necessity of repository-level context for test repair. In this case, the production method `execute()` was refactored to accept a new parameter, `stellarContext`, of type `Context`. Consequently, the existing test method, `testExecuteTransformation()`, which invokes `execute()`, becomes obsolete. Without modification, it fails with a compilation error due to the mismatched method signature.

Existing methods often fail to adequately incorporate repository-level context, which can lead to incorrect or incomplete code generation. For example, when constructing an instance of the `Context` class, LLM tends to use the default or common constructor (i.e., `Context.Builder.build()`) if no additional context is available. This may result in generation failures when the default constructor does not reflect the actual usage. However, this method is defined in a separate file, `Context.java`. This highlights a critical limitation

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

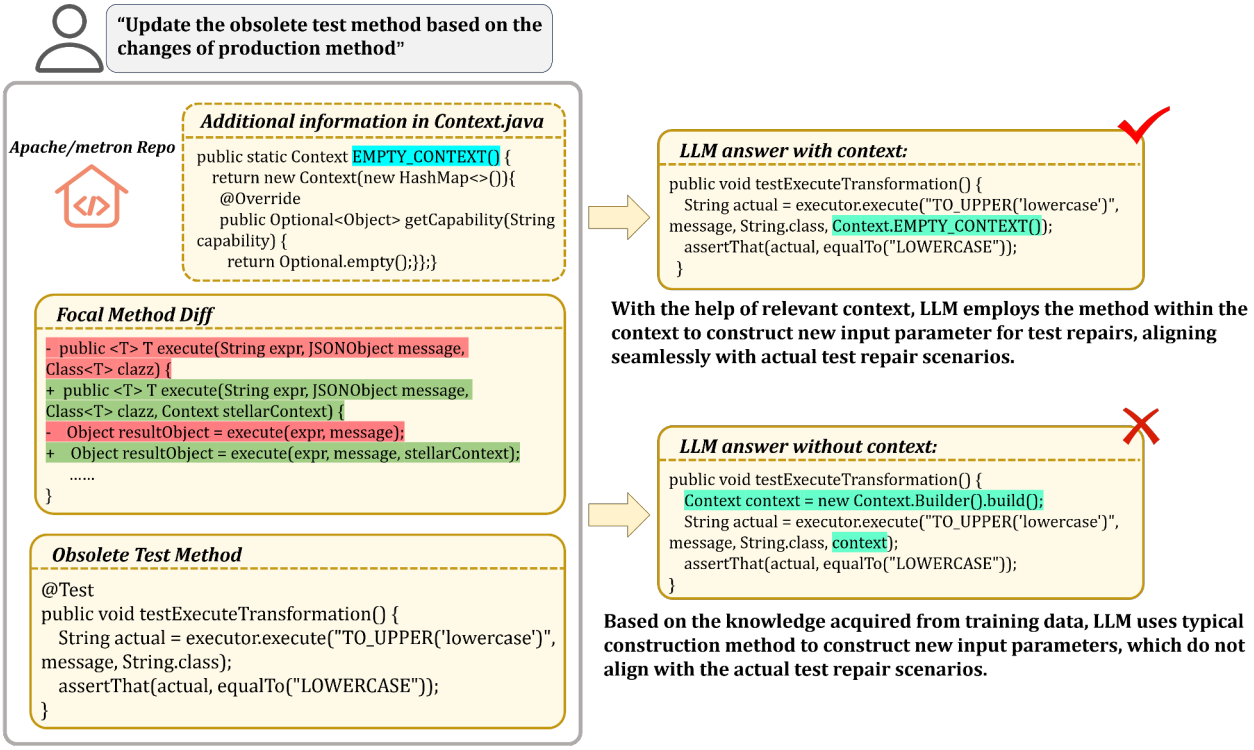

Figure 1: A motivating example from a commit of Apache/metron.

of approaches that rely solely on local context: they cannot discover non-local, yet essential, information required for a valid repair. Only by analyzing the definition of the `Context` class within the repository can the model identify the correct, idiomatic way to instantiate it.

This example reveals that the contextual information needed for test repair is often embedded within the dependencies of the evolved production code. The introduction of the `Context` parameter creates a direct dependency, and understanding this dependency's usage conventions is key to a successful repair. Existing methods that collect context based on fixed strategies may fail to collect these key pieces of information. Motivated by this observation, our approach is designed to systematically explore these dependencies. Starting from the focal method, TRACE traverses the invocation relationships within the project repository to retrieve relevant context from related components, whether they are local or distant. This repository-level exploration enables TRACE to gather the rich contextual information necessary for accurate and robust test repair, such as identifying the correct `EMPTY_CONTEXT()` method in our example.

To devise an effective automated approach for repairing obsolete test code, we first conducted an empirical investigation into where the essential contextual information for such repairs is typically located within a software repository. Our analysis of co-evolving product and test code revealed that the necessary context frequently resides within a limited scope of invocation relationships surrounding the focal method under test. This observation directly informed the design of our system, TRACE, which adopts a **repository-level** approach to retrieve the required context from the entire project repository. Specifically, TRACE systematically extracts and utilizes contextual details by analyzing repository-wide call relations, thereby enabling the generation of accurate and updated test code.

B. Empirical Insight on Context Distribution

To justify our repository-level retrieval strategy, we first conducted an empirical study to understand where the contextual information needed for test repair is typically located. We selected 50 test case repair instances from the benchmark dataset introduced by Hu et al. [11]. Each instance includes a modified focal method and its corresponding updated test method. To prevent bias, these instances are disjoint from those used in our final evaluation.

For each instance, we employed tree-sitter [43] to perform static analysis across the entire project repository. We identified the added method invocations in the updated test method and resolved their definitions to the corresponding files. To assess the relationship between these context elements and the focal method, we constructed a static call graph rooted at the focal method and recorded the call depth at which each file appeared. The call depth reflects the number of invocation steps required to reach a file containing contextual methods from the focal method's definition. For example, if a method in the same file as the focal method is referenced in the updated test, it is recorded at depth 0. If the focal method calls another method

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

located in a different file and the test update references that file, the context is assigned to depth 1. Deeper levels follow this pattern based on transitive invocations.

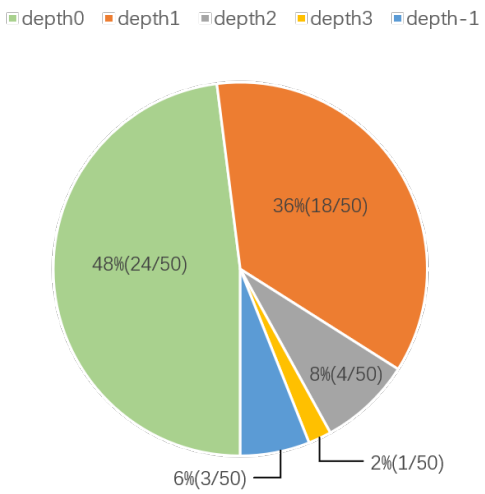

Figure 2: Distribution of Test Case Repair Context by Call Hierarchy Level.

Fig. 2 presents the distribution of contextual depths across the 50 analyzed cases. In 24 instances, all referenced contextual elements were located in the same file as the focal method (depth 0). In 18 cases, the test relied on context from files reachable via a single-level invocation (depth 1). Only a small number of instances required context from depth 2 or 3. One case could not be resolved due to the use of dynamic dispatch or decoupled design patterns that static analysis could not capture. The result indicates that most of the necessary context for test repair is confined within a narrow invocation scope. This locality supports the design of TRACE, which prioritizes efficient, repository-level static analysis and constrains context retrieval to relevant portions of the call graph.

C. Overview of TRACE

Based on the above insight, we present TRACE (Test Repair via Agent-guided Context Extraction), a framework designed to automatically repair obsolete test methods by leveraging repository-level contextual information. As illustrated in Fig. 3, given a modified focal method, its corresponding obsolete test method, and the full project repository as input, TRACE performs test repair through the following three stages:

1) **File-Level Call Graph Construction**: This stage takes the entire project repository as input and analyzes its source code to construct a file-level call graph. In this graph, nodes represent source files and edges indicate inter-file invocation relationships. The resulting graph serves as a structural foundation to constrain and guide subsequent context retrieval.
2) **Context Retrieval Agent**: Given the diff between the focal methods and the obsolete test method as the query input, this stage employs LLM-powered agent to iteratively retrieve relevant contextual information from the repository. The agent navigates the call graph and invokes specialized tools to gather only the necessary code snippets and declarations that may assist in repairing the test.
3) **Test Repair Generation**: Finally, the retrieved context is combined with the focal method's changes and the obsolete test code into a structured prompt. This comprehensive prompt is then fed to an LLM to generate an updated test method. This final stage aims to produce a repair that is not only syntactically correct but also semantically consistent with the evolved behavior of the production code.

D. File-Level Call Graph Construction

Context retrieval in large repositories demands a focused exploration of relevant code components. To efficiently guide the agent's search, TRACE constructs a file-level call graph representing invocation relationships between source files.

TRACE begins by mapping Java class names to their corresponding file paths throughout the repository. Leveraging the tree-sitter parsing framework [43], it statically analyzes the repository to extract `import` statements, `package` declarations, and method invocation information across classes. These extracted dependencies are resolved to generate edges between nodes in the call graph, where each node represents a source file and edges indicate invocation relationships at the file level. This file-level call graph facilitates targeted traversal starting from the focal method's file, allowing TRACE to identify context-bearing files that are directly or indirectly connected. For instance, in the motivating example shown in Fig. 1, the file `context.java` (highlighted in red in Fig. 4) contains the critical contextual information necessary for repairing the test method. TRACE retrieves the information from the File-Level Call Graph through the Context Retrieval Agent. The agent traverses the call relationship graph to gradually obtain the information in the call graph of the focal method. Ultimately, it is able to locate the key repair information in `context.java`.

E. Context Retrieval Agent

Test repair requires collecting relevant contextual information distributed across a large and complex codebase. Directly providing the entire repository to the LLM is infeasible due to input length limitations and the risk of overwhelming the model with irrelevant data. To address this, TRACE employs an LLM-powered agent that selectively and iteratively collects pertinent data from the repository.

The agent is initialized with a query containing the focal method's code changes and the obsolete test case. Leveraging the LLM's reasoning capabilities, it dynamically determines what additional context is necessary at each step. Instead of indiscriminately processing all repository content, the agent selectively interacts with three specialized retrieval tools: RETRIEVE CLASS INFO, RETRIEVE CALLER METHOD, and RETRIEVE TARGET FILE. This adaptive and iterative

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

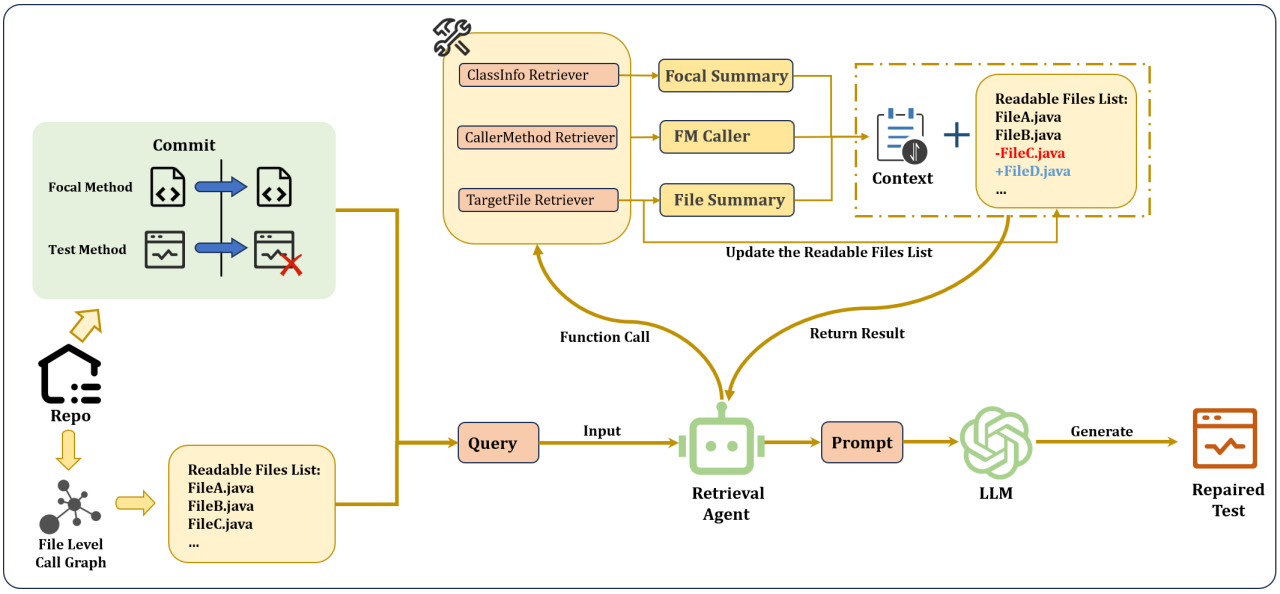

Figure 3: Overview of TRACE.

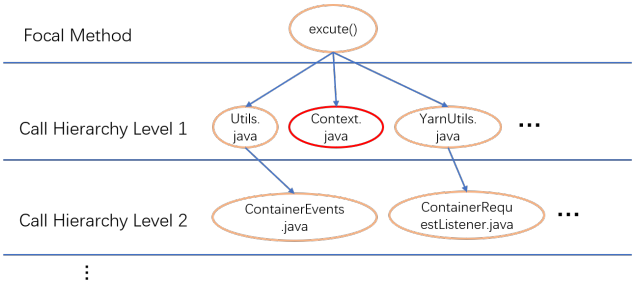

Figure 4: The File-Level Call Graph of *execute()*.

```
Class name: DefaultStellarExecutor
Variables:
    private Map<String, Object> state;
Methods:
    @Override public Map<String, Object> getState()
    @Override public void assign(String
    ↪  variable,String expression,JSONObject
    ↪  message,Context stellarContext)
    @Override public <T> T execute(String
    ↪  expr,JSONObject message,Class<T>
    ↪  clazz,Context stellarContext)
    @Override  public void clearState()
    private Object execute(String expr,JSONObject
    ↪  msg,Context stellarContext)
```

Figure 5: Class Info of *execute()*

retrieval strategy enables the agent to progressively refine its understanding of the relevant code regions. By constructing a focused, task-specific context window for each repair task, this mechanism enhances the relevance and quality of the test code updates generated by TRACE. The effectiveness of the LLM-guided retrieval process is empirically evaluated in **RQ3**, which measures its impact on test repair performance.

a) Retrieve Class Info: This tool inspects the class containing the focal method and returns a summary of its structure, including member fields and method signatures. This information helps identify changes like renamed or removed methods and modified fields, which are common causes of test obsolescence. For the case in Fig. 1, the summary provided by this tool is shown in Fig. 5, and corresponds to the "Focal Summary" in the architecture diagram (Fig. 3).

b) Retrieve Caller Method: The agent examines methods that invoke the focal method to understand how it is used in higher-level business logic. This context helps verify whether the existing test behavior remains semantically consistent with real usage, particularly when changes involve parameter handling, return values, or exception control. For the case in Fig. 1, the return result of the Retrieve Caller Method(i.e., the FM Caller in Fig. 3) is shown in Fig. 6.

```
@SuppressWarnings({"unchecked"})
public void run() throws
↪  YarnException,IOException,InterruptedException {
  Stirng ex1 = ExecuteStr;
  executor.execute(ex1,clazz,Context.EMPTY_CONTEXT())
    ...
}
```

Figure 6: Caller Method of *execute()*

c) Retrieve Target File: The agent explores additional files along the file-level call graph. It maintains a dynamic "readable files list" initially seeded with files directly invoked by the focal method. After reading a file, TRACE adds its direct callees to the list while removing the visited file. This iterative mechanism enables TRACE to progressively construct

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

a transitive dependency graph that spans cross-file relationships, ensuring that necessary context is collected while minimizing the inclusion of irrelevant information. The result of retrieving the file *Context.java* (i.e., File Summary in the Fig. 3) for the case in Fig. 1 is shown in Fig. 7.

```
Class name: Context
Variables:
    private Map<String, Capability> capabilityMap =
    ↪  new HashMap<>();
    private Map<String, Capability> capabilities;
Methods:
    public static Context EMPTY_CONTEXT()
    public Optional<Object> getCapability(Enum<?>
    ↪  capability)
    public Optional<Object> getCapability(String
    ↪  capability)
    public void addCapability(String s, Capability
    ↪  capability)
    public void addCapability(Enum<?> s, Capability
    ↪  capability)
```

Figure 7: Information of the target file *Context.java*

After performing one or more actions, the agent evaluates whether the currently retrieved context is sufficient to support test repair. If more information is needed, it continues retrieving additional context. Once the available information is deemed adequate, the agent assembles the collected data into a structured prompt and proceeds to the next stage for generating the repaired test code.

F. Test Repair Generation

Generating accurate and maintainable test updates requires effectively integrating the collected contextual information into the repair process. To achieve this, TRACE employs a structured prompt that guides the LLM to generate targeted repairs aligned with the evolved codebase. This prompt is carefully designed to provide the LLM with all the necessary information while ensuring that the output is focused and relevant. The prompt consists of several key components:

- A task description that positions the model as an expert in Java test maintenance.
- A scenario that includes the original test method (`original_testCode`), the diff of the modified functional method (`focal_diff`), and additional context extracted from the repository (`context`).
- An output requirement that instructs the model to return only the repaired test code without any explanations.

This structured prompt enables TRACE to synthesize updated test logic that reflects recent API changes, adjusts assertions, resolves obsolete method calls or field accesses, and preserves semantic consistency with the revised functional behavior. By iteratively focusing on the most relevant repair context, TRACE effectively balances precision and efficiency, enabling accurate and maintainable test updates across evolving software systems. To illustrate how this prompt works, we provide an example below. This example demonstrates how the prompt is structured and how it guides the LLM to perform the desired task:

```
Task Description:
You are an expert in Java code evolution
and test maintenance. Your task is to
update a test method when changes to the
corresponding functional method may have
caused the test to become outdated or
invalid.
Scenario:
You are given:
  • The original test method that require
    updates:
    {original_testCode}

  • A diff representing the recent changes
    made to the functional method:
    {focal_diff}

  • Additional context to support your
    reasoning and repair:
    {context}

Output Requirement:
  • Generate the updated test method that
    reflects the changes in the functional
    code.
  • Only return the repaired test code. Do
    not include explanations or commentary.
```

V. EXPERIMENT

In this section, we will introduce our choices regarding the benchmark dataset, baselines, and evaluation metrics used in our experiments.

A. Benchmark

To evaluate the effectiveness of TRACE, we adopt the dataset constructed by Liu et al. [12] as our benchmark. Following previous studies [9, 11], this dataset comprises 136 real-world co-evolution cases collected from 54 open-source Java projects on GitHub, each involving syntactic changes to the API interfaces of focal methods.

This benchmark is selected based on two main considerations. It originates from a well-established empirical dataset used in prior research, which supports the reliability and reproducibility of our evaluation. In addition, the test methods included in this benchmark exhibit a clear need for updates. Each case involves obsolete test code that no longer aligns with the corresponding focal method due to changes in API signatures, invocation patterns, or related data structures. Compared to the broader set of co-evolution cases, this subset has been specifically refined to emphasize scenarios where test repair is both necessary and non-trivial. These characteristics make it well suited for evaluating TRACE.

B. Baselines

To assess TRACE's effectiveness, we selected four baseline approaches for comparison: CEPROT [11], SYNTER [12], REACCEPT [33], and NaiveLLM. This selection was informed by our comprehensive survey of existing test repair methodologies.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

- CEPROT is a pioneering approach that identifies and automatically updates obsolete test cases based on focal method changes. It leverages CodeT5 model to learn from code changes and generate updated test cases. CEPROT has demonstrated high precision and recall in identifying and updating test cases, significantly improving test coverage.
- SYNTER aggregates three types of contextual information into a structured prompt to guide the LLM in generating repairs. It constructs context through static analysis and neural reranking. In our experiments, we excluded the source file segments containing the test method under repair from SYNTER's contextual input to focus on the model's repair capabilities.
- REACCEPT automates production and test code co-evolution by leveraging LLMs and dynamic validation. It uses experience-based prompt templates, dynamic validation, and retrieval-augmented generation techniques. REACCEPT has achieved high accuracy in identifying and updating obsolete test cases.
- NaiveLLM provides the LLM with only the focal method changes and the test code requiring repair, without additional context. Given the extensive training of LLMs like GPT-4 [22] and DeepSeek-Coder [23] on code-related tasks, they can perform test code repair even without extra context. This baseline evaluates the inherent capabilities of LLMs in test repair.

C. Metrics

Based on previous research, we designed evaluation metrics from the perspectives of text matching and test result consistency.

- **CodeBLEU:** CodeBLEU [44] is derived from the BLEU (Bilingual Evaluation Understudy) metric used in machine translation. It measures the similarity of generated code to reference code by considering both syntactic and semantic aspects. CodeBLEU has been widely adopted for evaluating code generation tasks. Therefore, we use CodeBLEU to assess the similarity between the generated test method and the ground-truth test method.
- **Accuracy:** This metric indicates the proportion of samples in which the generated test method is textually identical to the actual repaired test method.
- **Syntactic Pass Rate (SPR)** This metric measures the proportion of generated test cases that are syntactically valid. Specifically, a test case is considered to pass if it can be successfully parsed by the Java parser without triggering syntax errors.
- **Compilation Pass Rate (CPR)**: This metric evaluates whether the generated test cases can be compiled successfully within the context of the project. In addition to syntactic correctness, CPR requires that all type references, method invocations, and class dependencies are correctly resolved.
- **Test Result Consistency:** This metric checks whether the execution results of the generated test code are consistent with those of the actual repaired test code.

VI. EVALUATION

Based on the aforementioned experimental design, we systematically evaluate TRACE's effectiveness and conduct a comprehensive analysis of the results. Our investigation addresses the following key research questions (RQs):

- **RQ1:(Effectiveness of TRACE)** Can TRACE effectively repair obsolete tests effectively?
- **RQ2:(Effectiveness of Retrieval Agent)** How effective is the retrieval agent in assisting LLM in repairing test code?
- **RQ3:(Necessity and Component Contribution of the Retrieval Agent)** Is the agent-based retrieval mechanism essential for TRACE's performance, and how does each individual component of the retrieval agent contribute to the overall effectiveness of test repair?

A. RQ1: Effectiveness of TRACE

To evaluate the effectiveness of TRACE in repairing obsolete test cases, we conduct a comprehensive comparison against four baselines: CEPROT, SYNTER, REACCEPT, and **NaiveLLM**. The evaluation focuses on three key dimensions: Compilability, Intent Preservation, and Behavioral Correctness. The results are summarized in Table I.

a) Compilability.: We assess whether the generated test cases are syntactically valid and successfully compile against the updated production code. We use two metrics for this dimension: **Syntactic Pass Rate (SPR)** and **Compilation Pass Rate (CPR)**. TRACE, along with SYNTER, REACCEPT, and NaiveLLM, achieves a perfect SPR of 100%, highlighting the general reliability of LLM-based approaches in producing well-formed code. In contrast, CEPROT, which is based on a fine-tuned CodeT5 model, achieves a significantly lower SPR of 48.2%, suggesting limited adaptability to diverse test repair scenarios. In terms of CPR, TRACE achieves 97.1%, closely matching SYNTER (96.2%) and outperforming REACCEPT (90.3%) and NaiveLLM (88.7%), indicating that the generated tests are not only syntactically correct but also compatible with the updated codebase.

b) Intent Preservation.: We assess how well the generated test cases preserve the original test intent. This is approximated using **CodeBLEU**, which reflects semantic similarity to reference implementations. TRACE achieves a CodeBLEU score of 84.97, which is comparable to SYNTER (83.78) and superior to both REACCEPT (82.66) and NaiveLLM (83.67). These results suggest that TRACE is effective at preserving the intended test logic, even as the underlying code evolves.

c) Behavioral Correctness.: We assess whether the generated test cases exhibit correct runtime behavior when executed on the updated system. We use **Accuracy** to measure the proportion of fully correct test repairs, and **Consistency** to assess the stability of test outputs across repeated executions. TRACE achieves an Accuracy of 38.24%, closely approaching SYNTER (38.97%) and outperforming REACCEPT and NaiveLLM. Moreover, TRACE obtains the highest Consistency score of 80.90%, indicating its robustness in generating behaviorally correct and stable test repairs.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

Table I: Effectiveness of repairing obsolete test case.

Approach	Compilability		Intent Preservation	Behavioral Correctness	
	SPR (%)	CPR (%)	CodeBLEU	Accuracy (%)	Consistency (%)
CEPROT	48.2%	33.3%	72.91	4.40%	19.85%
SYNTER	**100%**	96.2%	83.78	33.82%	77.21%
REACCEPT	**100%**	90.3%	82.66	29.41%	70.59%
NAIVELLM	**100%**	88.7%	83.67	30.88%	69.85%
TRACE	**100%**	**97.1%**	**84.97**	**38.24%**	**80.88%**

Table II: Retrieval Quality of Context Agent (Human-Evaluated on 50 Samples)

Metric	Score (%)	Description
Coverage	72.00	Proportion of required context correctly retrieved
Noise	38.67	Proportion of irrelevant tokens in the retrieved context

B. RQ2: Effectiveness of Retrieval Agent

To evaluate the effectiveness of the retrieval agent in TRACE, we conduct a human evaluation focused on its ability to gather necessary information for test repair. We randomly sampled 50 cases from the benchmark and manually identified the minimal required context elements (e.g., method definitions, field declarations, and call sites) that are essential for correct test repair. This serves as the golden reference. We then compare these with the context retrieved by TRACE's agent.

The evaluation uses two metrics: (1) **Coverage**, which measures the percentage of golden elements present in the retrieved context; and (2) **Noise**, which reflects the proportion of irrelevant content within the retrieved context. As shown in Table II, the retrieval agent achieves an average coverage of 72.0% and a noise ratio of 38.67%.

These results indicate that the agent can capture a substantial portion of the necessary context, though some irrelevant or redundant content is also introduced. Despite the noise, the high performance of the final repair results demonstrates that the LLM is generally robust to a moderate level of irrelevant input. This further supports the effectiveness of using the retrieval agent to guide test repair.

C. RQ3: Necessity and Component Contribution of the Retrieval Agent

This research question aims to evaluate whether the retrieval agent is essential for effective test repair, and to analyze the contribution of each component within the agent.

To assess the necessity of the retrieval agent, we compare TRACE with a variant that directly concatenates all retrieved context and feeds it to the LLM without agent-guided orchestration (denoted as *w/o agent*). This variant bypasses the agent's structured reasoning and instead supplies raw context in bulk. As shown in Table III, TRACE significantly outperforms the *w/o agent* variant in terms of CodeBLEU, Accuracy, and Consistency metrics. This demonstrates that simply providing more information is insufficient; without the agent's targeted and iterative retrieval strategy, irrelevant or noisy inputs may overwhelm the LLM and impair its ability to generate accurate repairs. The results highlight the critical role of the retrieval agent in focusing the LLM on relevant context, thereby enhancing semantic alignment and behavioral correctness of the repaired test code.

Table III: Necessity of the Retrieval Agent

Approach	CodeBLEU	Accuracy	Consistency
w/o agent	78.03	22.06%	65.44%
TRACE	**84.97**	**38.24%**	**80.88%**

To further understand the effectiveness of each retrieval component, we conduct an ablation study by selectively disabling individual actions within the agent: (1) *Retrieve Target File*, (2) *Retrieve Class Info*, and (3) *Retrieve Caller Method*. These actions correspond to collecting different types of contextual information critical to test repair. The results are summarized in Table IV.

To evaluate the impact of each component, we conducted a significance analysis using paired t-tests on the raw data, comparing the full TRACE model against configurations with each component removed. The analysis involved calculating the difference in CodeBLEU scores for each of the 136 test cases between the full model and each ablated configuration, followed by computing the t-statistic and p-value to test the null hypothesis that removing a component does not affect CodeBLEU performance. A single-tailed test was used, as we hypothesized that the full model outperforms the ablated versions. To account for multiple comparisons across the three components, we applied Bonferroni correction, adjusting the significance level to $\alpha' = 0.05/3 \approx 0.0167$. The results confirm that removing any component significantly reduces CodeBLEU, with all p-values below the adjusted threshold. Specifically, disabling *Retrieve Target File* causes the largest decline, with CodeBLEU decreasing from 84.97 to 83.35, yielding a t-statistic of 2.6674 and a p-value of 0.0043, indicating that access to the target file is critical for semantically accurate repairs. Omitting *Retrieve Class Info* reduces CodeBLEU to 83.18, with a t-statistic of 2.8160 and a p-value of 0.0028, demonstrating that class-level structural and contextual knowledge enhances repair precision. Similarly, disabling *Retrieve Caller Method* lowers CodeBLEU to 83.12, with a t-statistic of 3.5660 and a p-value of 0.0003, highlighting the importance of caller-side context for capturing implicit usage patterns essential for effective repairs. For reference, removing these components also impacts Accuracy and Consistency, with Accuracy decreasing from 38.24% to 31.62%, 33.09%, and 30.15%, respectively,

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restriction

and Consistency from 80.9% to 75.1%, 75.3%, and 75.1%, respectively, underscoring their broader importance. These findings validate that each component significantly contributes to the model's CodeBLEU performance.

These findings confirm that the retrieval agent's components each contribute complementary information, and their combined use enables TRACE to generate accurate and consistent test repairs by leveraging a well-rounded understanding of the codebase.

Table IV: Result of Ablation Experiment

Approach	CodeBLEU	Accuracy (%)	Consistency (%)
TRACE (w/o targetFile)	83.35	31.62%	75.1%
TRACE (w/o focalClass)	83.18	33.09%	75.3%
TRACE (w/o callerMehod)	83.12	30.15%	75.1%
TRACE	**84.97**	**38.24%**	**80.9%**

VII. DISCUSSION

A. Failure Analysis

To better understand the limitations of our approach, we conduct a failure analysis on the test cases where TRACE failed to generate correct repairs. Specifically, we focus on instances from Table I where the generated tests exhibited inconsistent execution results or had notably low CodeBLEU scores. Through manual inspection of these cases, we identify the following three primary causes of failure:

- **High complexity of focal method changes.** In some cases, the target focal method underwent substantial modifications, including changes in its logic, input-output structure, or side effects. Such changes often require a deep understanding of the new semantics, which can be difficult for the model to infer solely based on retrieved context.
- **Incorrect contextual guidance.** In several cases, the retrieval agent introduced misleading examples. For instance, if the retrieved code snippet invokes the focal method using parameters constructed via a specific static utility method, the LLM tends to adopt the same pattern in the generated test. However, this may not align with the actual updated test, which uses a different parameter construction mtehod, leading to inconsistencies.
- **Missing test-related context.** In many failure cases, the updated test class introduces additional setup code, such as newly declared fields or helper methods, to support the repaired test method. Since this auxiliary context is not included in the prompt provided to the LLM, the LLM lacks the necessary information to correctly reconstruct the expected test logic, resulting in semantically divergent or incomplete repairs.

These findings indicate that the accuracy of TRACE depends heavily on the quality and completeness of the provided context. Improvements in change comprehension, filtering out misleading retrievals, and enriching the prompt with relevant structural information represent promising directions for future research.

B. Limitations of TRACE

Although TRACE demonstrates promising performance in repairing obsolete test cases, it also faces several limitations. The effectiveness of the approach is closely tied to the quality and completeness of the retrieved context. Our ablation and failure analyses suggest that when critical contextual elements, such as caller methods or changes in auxiliary classes, are absent, the repair performance of TRACE can degrade substantially. This issue is partly attributable to the use of static call graph construction, which often fails to capture dynamic dispatch behavior and reflection-based invocations that are prevalent in Java systems.

While the file-level call graph design helps reduce the complexity of context retrieval, it can still introduce irrelevant or noisy dependencies, particularly in systems where classes exhibit broad or loosely coupled interactions. In such cases, TRACE may retrieve superfluous information that misleads the language model, leading to inaccurate or suboptimal repairs. Moreover, the current strategy relies on the language model itself to decide whether further context should be retrieved, which does not always reflect the actual information needs of the test repair task. Incorporating more principled decision-making mechanisms or learning-based retrieval policies may enhance the effectiveness and reliability of the approach.

C. Data Privacy and Leakage Risk

Data leakage poses a potential risk when using LLMs for test code repair, especially when sensitive project code is inadvertently exposed through external APIs or fine-tuned local deployments [45, 46, 47]. Although experimental results show that NaiveLLM can correctly repair some tests with data leakage, it does not affect the overall effectiveness evaluation of TRACE. However, data leakage may still lead to overestimation of repair quality and inflated performance metrics. To mitigate these risks, future work should focus on secure LLM deployment strategies, context sanitization, and differential privacy techniques to ensure privacy preservation and reliable repair outcomes [48, 49].

D. Challenges in Test Repair Evaluation

Evaluating test code generation remains a complex and unresolved challenge. Although we use metrics such as SPR, CPR, CodeBLEU, Accuracy, and Consistency to assess the generated outputs, each comes with inherent limitations. CodeBLEU captures syntactic and semantic similarity but does not account for functional correctness. Accuracy, defined as an exact match with the reference test, applies overly strict criteria and may penalize functionally equivalent variants. Consistency focuses on behavioral alignment but relies on the availability and reliability of test execution environments, which can differ across projects. To address these limitations, we conducted a manual evaluation of the test cases repaired by TRACE, and the results were largely in agreement with those produced by the automated metrics.

Real-world test repair often permits multiple valid solutions. Under such circumstances, strict reference-based evaluation

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

may fail to reflect the true quality of the repair. Evaluation frameworks could be improved by incorporating human judgment, test coverage analysis, and mutation testing to provide a more comprehensive assessment of repair effectiveness.

VIII. Conclusions

We proposed TRACE, a novel approach for repairing obsolete test code by leveraging LLMs and agent-based context retrieval. TRACE identifies the focal method changes and systematically extracts contextual information through three targeted retrieval actions: *Retrieve Class Info*, *Retrieve Caller Method*, and *Retrieve Target File*. By constructing a file-level call graph, TRACE balances context completeness and prompt manageability, enabling efficient and accurate test code repair. Our evaluation on real-world obsolete test cases demonstrates that TRACE outperforms existing baselines. The ablation studies confirm that each retrieval action contributes significantly to the effectiveness of test repair, while failure analysis highlights the challenges posed by complex focal changes, misleading contexts, and untracked auxiliary test setup modifications.

Despite its effectiveness, TRACE has limitations related to static analysis precision, privacy concerns when interfacing with LLMs, and the inherent difficulty of evaluating test correctness. These findings suggest multiple directions for future research, including enhancing retrieval strategies with dynamic analysis, securing context sharing for model inference, and developing more comprehensive test evaluation frameworks.

References

[1] Václav Rajlich. "Software evolution and maintenance". In: *Future of Software Engineering Proceedings*. 2014, pp. 133–144.

[2] Ned Chapin et al. "Types of software evolution and software maintenance". In: *Journal of software maintenance and evolution: Research and Practice* 13.1 (2001), pp. 3–30.

[3] Krishan Kumar Aggarwal. *Software engineering*. New Age International, 2005.

[4] Glenford J Myers, Corey Sandler, and Tom Badgett. *The art of software testing*. John Wiley & Sons, 2011.

[5] Paul Ammann and Jeff Offutt. *Introduction to software testing*. Cambridge University Press, 2016.

[6] Leandro Sales Pinto, Saurabh Sinha, and Alessandro Orso. "Understanding myths and realities of test-suite evolution". In: *Proceedings of the ACM SIGSOFT 20th international symposium on the foundations of software engineering*. 2012, pp. 1–11.

[7] Ermira Daka and Gordon Fraser. "A survey on unit testing practices and problems". In: *2014 IEEE 25th International Symposium on Software Reliability Engineering*. IEEE. 2014, pp. 201–211.

[8] Cosmin Marsavina, Daniele Romano, and Andy Zaidman. "Studying fine-grained co-evolution patterns of production and test code". In: *2014 IEEE 14th International Working Conference on Source Code Analysis and Manipulation*. IEEE. 2014, pp. 195–204.

[9] Sinan Wang et al. "Understanding and facilitating the co-evolution of production and test code". In: *2021 IEEE International conference on software analysis, evolution and reengineering (SANER)*. IEEE. 2021, pp. 272–283.

[10] Yue Wang et al. "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation". In: *arXiv preprint arXiv:2109.00859* (2021).

[11] Xing Hu et al. "Identify and update test cases when production code changes: A transformer-based approach". In: *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE. 2023, pp. 1111–1122.

[12] Jun Liu et al. "Fix the Tests: Augmenting LLMs to Repair Test Cases with Static Collector and Neural Reranker". In: *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE. 2024, pp. 367–378.

[13] Mosh Levy, Alon Jacoby, and Yoav Goldberg. "Same task, more tokens: the impact of input length on the reasoning performance of large language models". In: *arXiv preprint arXiv:2402.14848* (2024).

[14] Zhongwei Wan et al. "Look-m: Look-once optimization in kv cache for efficient multimodal long-context inference". In: *arXiv preprint arXiv:2406.18139* (2024).

[15] Yupeng Chang et al. "A survey on evaluation of large language models". In: *ACM transactions on intelligent systems and technology* 15.3 (2024), pp. 1–45.

[16] Mengchao Ren. "Advancements and Applications of Large Language Models in Natural Language Processing: A Comprehensive Review". In: *Applied and Computational Engineering* 97 (2024), pp. 55–63.

[17] Juyong Jiang et al. "A survey on large language models for code generation". In: *arXiv preprint arXiv:2406.00515* (2024).

[18] Qing Xue. "Unlocking the potential: A comprehensive exploration of large language models in natural language processing". In: *Applied and Computational Engineering* 57 (2024), pp. 247–252.

[19] Lishui Fan, Mouxiang Chen, and Zhongxin Liu. "Self-Explained Keywords Empower Large Language Models for Code Generation". In: *arXiv preprint arXiv:2410.15966* (2024).

[20] Bo Lin et al. "Cct5: A code-change-oriented pre-trained model". In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2023, pp. 1509–1521.

[21] Yihao Qin et al. "Agentfl: Scaling llm-based fault localization to project-level context". In: *arXiv preprint arXiv:2403.16362* (2024).

[22] Josh Achiam et al. "Gpt-4 technical report". In: *arXiv preprint arXiv:2303.08774* (2023).

[23] Daya Guo et al. "Deepseek-coder: When the large language model meets programming–the rise of code intelli-

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio

gence, 2024". In: *URL https://arxiv. org/abs/2401.14196* 5 (2024), p. 19.

[24] B Steenhoek et al. "A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection." In: *arXiv preprint arXiv:2403.17218* (2024).

[25] Angela Fan et al. "Large language models for software engineering: Survey and open problems". In: *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE. 2023, pp. 31–53.

[26] Claus Klammer, Georg Buchgeher, and Albin Kern. "A retrospective of production and test code co-evolution in an industrial project". In: *2018 IEEE Workshop on Validation, Analysis and Evolution of Software Tests (VST)*. IEEE. 2018, pp. 16–20.

[27] Weifeng Sun et al. "Revisiting the identification of the co-evolution of production and test code". In: *ACM Transactions on Software Engineering and Methodology* 32.6 (2023), pp. 1–37.

[28] Quentin Le Dilavrec et al. "Untangling spaghetti of evolutions in software histories to identify code and test co-evolutions". In: *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE. 2021, pp. 206–216.

[29] Andy Zaidman et al. "Studying the co-evolution of production and test code in open source and industrial developer test processes through repository mining". In: *Empirical Software Engineering* 16.3 (2011), pp. 325–364.

[30] Leon Moonen et al. "The interplay between software testing and software evolution". In: *Software Evolution*. Springer, 2008, pp. 173–202.

[31] Sebastian Elbaum, David Gable, and Gregg Rothermel. "The impact of software evolution on code coverage information". In: *Proceedings of the International Conference on Software Maintenance (ICSM)*. IEEE. 2001, pp. 170–179.

[32] Gustavo Sizilio Nery, Daniel Alencar da Costa, and Uirá Kulesza. "An empirical study of the relationship between continuous integration and test code evolution". In: *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE. 2019, pp. 426–436.

[33] Jianlei Chi et al. "REACCEPT: Automated Co-evolution of Production and Test Code Based on Dynamic Validation and Large Language Models". In: *arXiv preprint arXiv:2411.11033* (2024).

[34] Rangeet Pan et al. "ASTER: Natural and Multi-language Unit Test Generation with LLMs". In: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE. 2024.

[35] Saranya Alagarsamy et al. "Enhancing Large Language Models for Text-to-Testcase Generation". In: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE. 2024.

[36] Lin Yang et al. "Optimizing Search-Based Unit Test Generation with Large Language Models: An Empirical Study". In: *Proceedings of the 15th Asia-Pacific Symposium on Internetware*. ACM. 2024.

[37] Lenne Hirata et al. "Software Test Generation Incorporating System Specification Information through Large Language Models and Adoption Possibility Evaluation of the Generated Results". In: *Proceedings of the 38th Annual Conference of the Japanese Society for Artificial Intelligence (JSAI)*. The Japanese Society for Artificial Intelligence. 2024.

[38] Wenhan Wang et al. "TESTEVAL: Benchmarking Large Language Models for Test Case Generation". In: *arXiv preprint arXiv:2406.04531* (2024).

[39] Arghavan Moradi Dakhel et al. "Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing". In: *arXiv preprint arXiv:2308.16557* (2023).

[40] Junjie Wang et al. "Software Testing with Large Language Models: Survey, Landscape, and Vision". In: *arXiv preprint arXiv:2307.07221* (2023).

[41] Quanjun Zhang et al. "TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models". In: *arXiv preprint arXiv:2409.17561* (2024).

[42] Chen Yang et al. "Enhancing llm-based test generation for hard-to-cover branches via program analysis". In: *arXiv preprint arXiv:2404.04966* (2024).

[43] tree-sitter. https://tree-sitter.github.io/tree-sitter/.

[44] Shuo Ren et al. "Codebleu: a method for automatic evaluation of code synthesis". In: *arXiv preprint arXiv:2009.10297* (2020).

[45] Matthew Rosenblatt et al. "Data leakage inflates prediction performance in connectome-based machine learning models". In: *Nature Communications* 15.1 (2024), p. 1829.

[46] Alexandre Matton et al. "On leakage of code generation evaluation datasets". In: *arXiv preprint arXiv:2407.07565* (2024).

[47] José Antonio Hernández López et al. "On inter-dataset code duplication and data leakage in large language models". In: *IEEE Transactions on Software Engineering* (2024).

[48] Medha Palavalli, Amanda Bertsch, and Matthew R Gormley. "A taxonomy for data contamination in large language models". In: *arXiv preprint arXiv:2407.08716* (2024).

[49] Md Rafi Ur Rashid et al. "Forget to flourish: Leveraging machine-unlearning on pretrained language models for privacy leakage". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 19. 2025, pp. 20139–20147.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:27:02 UTC from IEEE Xplore. Restrictio